49. ‘Benim Makinemde Çalışıyordu’: Docker ve Konteyner Mantığı

Yazılım Dünyasının En Meşhur Mazeretini Tarihe Gömen Konteyner Devrimi
Bölüm 9 • Zorluk Derecesi: Orta Seviye • Tahmini Okuma Süresi: 12 dk

---

> 🎙️ **NotebookLM & Audio Overview Podcast Notları**
>
> **Podcast Tartışma Sorusu:**
> *“Bir junior mühendisin ‘Ama benim bilgisayarımda çalışıyordu!’ demesi neden bir senior lider için en sinir bozucu cümledir ve Docker bunu nasıl çözer?”*
>
> **Bölümden Çıkarılacak 3 Altın Cümle:**
> - 📌 Docker, uygulamanızı tüm bağımlılıkları, işletim sistemi kütüphaneleri ve konfigürasyonlarıyla birlikte izole bir kutuya (Container) paketler. - 📌 İmaj (Image) mimari plandır (class); Konteyner (Container) ise o planın çalışan canlı örneğidir (instance). - 📌 Çok aşamalı derleme (Multi-Stage Build), üretim imajlarını 1 GB’tan 50 MB’a düşürerek hem güvenlik hem hız sağlar.

📖 1. Giriş Hikayesi & Günlük Hayat Analojisi

1950’li yıllardan önce uluslararası deniz ticaretini hayal edin.

Muzlar ahşap kasalara konur, pamuklar çuvallara basılır, viski şişeleri samanların arasına sarılırdı. Bir limandan diğerine aktarılırken şişeler kırılır, muzlar çürür, her limanın vinci farklı olduğu için yükleme günlerce sürerdi!

Sonra Malcolm McLean adında bir adam çıktı ve **Standart Çelik Taşımacılık Konteynerini** icat etti! İçinde ne olursa olsun (araba, muz, bilgisayar), dışındaki çelik kutunun boyutları ve kilitleri standarttır. Dünyadaki her vinç, her gemi, her tır bu konteyneri aynı hızla taşır.

Docker işte o çelik konteynerdir!

Uygulamanızı Docker kutusuna koyduğunuzda; kodunuzun Mac’te mi, Windows’ta mı yoksa AWS Ubuntu sunucusunda mı çalıştığının hiçbir önemi kalmaz. Her yerde aynı milisaniyede çalışır.

🧠 2. AI Çağında Yeni Zihniyet Modeli

Docker Kavramlar Sözlüğü:

```
1. DOCKERFILE:
   Konteynerin yemek tarifidir. "Hangi Linux sürümü, hangi Node.js, hangi dosyalar
kopyalanacak?"

2. DOCKER IMAGE (İmaj):
   Tarife göre fırından çıkmış dondurulmuş kek kalıbıdır. Değiştirilemez
(Immutable), taşınabilir.

3. DOCKER CONTAINER (Konteyner):
   İmajın RAM üzerinde çalışan canlı halidir. Başlatılabilir, durdurulabilir,
çoğaltılabilir.

4. DOCKER REGISTRY (Docker Hub / AWS ECR):
   İmajların internette saklandığı ve paylaşıldığı kütüphanedir.
```

🤖 3. AI Prompt Laboratuvarı (Kopyala-Yapıştır Hazır Prompt)

Aşağıdaki prompt şablonunu Cursor, Claude Code, ChatGPT veya Copilot ortamınızda doğrudan kullanabilirsiniz:

```
# ROL: DevOps ve Konteynerleştirme Uzmanı
Next.js ve Node.js 20 kullanan bir web uygulaması için production-ready `Dockerfile`
yazacağız.

## GÖREV:
1. 'Multi-Stage Build' (Çok Aşamalı Derleme) kullanarak derleme araçlarını nihai
imajdan temizle.
2. `node_modules` ve kaynak kodları non-root (root olmayan) güvenli bir kullanıcı
ile çalıştır.
3. İmaj boyutunu en aza indiren hafif `node:20-alpine` tabanını kullan.
```

💻 4. Kod İncelemesi: Acemi / Halüsinasyon vs. Üretim Kalitesi

❌ Acemi / Kontrolsüz AI Çıktısı (Riskli):

```
# Kötü / Ağır Dockerfile: 1.2 GB boyut, root kullanıcısı, devDependencies
temizlenmemiş!
FROM node:20
WORKDIR /app
COPY . .
RUN npm install
RUN npm run build
CMD ["npm", "start"] # Root olarak çalışır (BÜYÜK GÜVENLİK RİSKİ)
```

✅ Kıdemli Mühendis Tarafından İyileştirilmiş Sürüm (Güvenli & Temiz):

```
# Çok Aşamalı (Multi-Stage) Üretim Dockerfile

# AŞAMA 1: Bağımlılıkları Kur
FROM node:20-alpine AS deps
WORKDIR /app
COPY package.json package-lock.json ./
RUN npm ci

# AŞAMA 2: Uygulamayı Derle
FROM node:20-alpine AS builder
WORKDIR /app
COPY --from=deps /app/node_modules ./node_modules
COPY . .
RUN npm run build

# AŞAMA 3: Hafif & Güvenli Üretim Ortamı (Minimal Runner)
FROM node:20-alpine AS runner
WORKDIR /app
ENV NODE_ENV=production

# Güvenlik Kuralı: Asla 'root' kullanıcısı ile çalıştırma!
RUN addgroup --system --gid 1001 nodejs && \
    adduser --system --uid 1001 nextjs

# Sadece derlenmiş çıktıları kopyala (İmaj boyutu 1.2 GB yerine 80 MB!)
COPY --from=builder /app/public ./public
COPY --from=builder --chown=nextjs:nodejs /app/.next/standalone ./
COPY --from=builder --chown=nextjs:nodejs /app/.next/static ./.next/static

USER nextjs
EXPOSE 3000
ENV PORT=3000

CMD ["node", "server.js"]
```

💡 **Kod Analizi:** Multi-stage yapı sayesinde derleyici ve dev paketler `builder` aşamasında çöpe atılmış, sadece saf JavaScript ve asset’ler `runner` aşamasına kopyalanarak imaj boyutu 1.2 GB’tan 80 MB’a düşürülmüş ve `USER nextjs` ile root çalıştırma riski önlenmiştir.

⚠️ 5. Junior Tuzakları & Kıdemli Bakış Açısı

- **Tuzak:** Konteyneri varsayılan `root` kullanıcısı ile çalıştırıp sunucuda güvenlik açığı yaratmak.
  - **Çözüm:** Her zaman `USER node` veya özel bir non-root kullanıcı tanımlayın.
- **Tuzak:** `.dockerignore` dosyası oluşturmayıp `node_modules` ve `.env` dosyalarını imaja kopyalamak.
  - **Çözüm:** Proje köküne mutlaka kapsamlı bir `.dockerignore` dosyası ekleyin.
- **Tuzak:** Konteyner içinde veri saklayıp konteyner silindiğinde tüm veritabanını kaybetmek.
  - **Çözüm:** Kalıcı veriler için daima Docker Volumes (`-v db_data:/var/lib/postgresql/data`) kullanın.

🎬 6. Udemy Video Dersi Taslağı & Slayt Maddeleri

📽️ Ders Başlığı: *Ders 49: Docker 101 - ‘Benim Makinemde Çalışıyordu’ Efsanesine Son*

- 🖥️ **Slayt 1:** Neden ‘Benim makinemde çalışıyor’ mazereti artık kabul edilemez?
- 🖥️ **Slayt 2:** Konteyner vs Sanal Makine (VM): Hafifliğin sırrı.
- 🖥️ **Slayt 3:** Multi-Stage Build ile 1.2 GB’lık imajı 80 MB’a düşürmek.
- 🖥️ **Slayt 4:** Canlı Demo: Sıfırdan Dockerfile yazıp imaj derlemek.

Canlı Kodlama / Ekran Kaydı Senaryosu:
Bir Next.js projesinde `docker build -t my-app .` çalıştırın. `docker images` çıktısında imaj boyutunun 80 MB olduğunu gösterin. `docker run -p 3000:3000 my-app` ile tarayıcıda uygulamanın ayağa kalktığını canlı gösterin.

🎯 7. Günün Uygulamalı Görevi & Mini Sınav

🛠️ Günün Görevi:

Kişisel bir projeniz için çok aşamalı (multi-stage) bir `Dockerfile` ve `.dockerignore` dosyası yazarak yerel makinenizde imajı derleyin.

🧠 Bölüm Sonu Kontrol Testi:

Soru 1: Docker’da ‘Multi-Stage Build’ (Çok Aşamalı Derleme) kullanmanın en büyük faydası nedir? - A) Bilgisayara yeni monitör bağlamak - B) Derleme aşamasında kullanılan ağır araçları (compiler,

TypeScript, npm cache) üretim imajından çöpe atarak nihai imajı hem çok hafif (MB düzeyinde) hem de saldırılara karşı daha güvenli kılmak - C) Sadece C++ dilinde çalışması - D) İnternet gerektirmemesi > **Doğru Yanıt:** B > **Açıklama:** Multi-stage builds, gereksiz kaynak kodları ve derleme araçlarını nihai imajın dışında bırakarak hafif ve güvenli konteynerler üretir.

Soru 2: Docker konteynerleri ile geleneksel Sanal Makineler (Virtual Machine - VM) arasındaki temel fark nedir? - A) Hiçbir farkı yoktur - B) Sanal makineler her biri için ayrı bir misafir işletim sistemi (Guest OS) kurarken; Docker konteynerleri ana makinenin Linux çekirdeğini (Kernel) ortaklaşa kullanarak saniyeler içinde ayağa kalkar - C) Docker sadece mobilde çalışır - D) VM’lerin ücretsiz olması > **Doğru Yanıt:** B > **Açıklama:** Docker ana çekirdeği (Host Kernel) paylaştığı için çok daha az RAM harcar ve milisaniyeler içinde başlar.